CONTROLE DE VERSÕES

Git na prática

Tema	Git: repositório local e remoto, commits, branches, Pull Requests, merge e conflitos
Pré-requisito	Git instalado (versão 2.28 ou superior), editor de texto e conta no GitHub
Formato	Individual, com uma atividade final em dupla
Entrega	Link do repositório público no GitHub

> **Objetivo:** criar um repositório Git local e publicá-lo no GitHub; entender as três áreas do Git (diretório de trabalho, staging area e repositório); trabalhar com a política de branches master/develop/feature; integrar branches por Pull Request, comparando os tipos de merge (fast-forward, merge commit e squash); e provocar e resolver um conflito de merge.

Antes de começar

Você vai versionar um arquivo de texto, o `cancao.txt`, em vez de código, para que a atenção fique nos comandos do Git. A letra fornecida para a atividade é só material de trabalho: cada branch altera um trecho dela, e duas alteram a mesma linha de propósito, para gerar um conflito. Tudo funciona igual com arquivos .cs, .ts ou qualquer outro.

As primeiras etapas acontecem só no seu computador. A partir da Etapa 5, o repositório vai para o GitHub e as integrações passam a ser feitas por Pull Request. É por lá que o professor vai acompanhar o seu trabalho.

Mapa do roteiro

Etapas	O que você vai aprender

1 e 2	Configurar o Git e criar um repositório local
3 e 4	Fazer commits, entender a staging area, ver e desfazer mudanças
5	Publicar o repositório no GitHub
6 e 7	Criar branches e fazer o primeiro merge (fast-forward)
8 a 11	Trabalhar em paralelo e integrar por Pull Request: merge commit, conflito e squash
12	Publicar uma versão na master com tag

> **TRÊS COMANDOS PARA USAR SEMPRE QUE TIVER DÚVIDA**
>
> `git status`: em que branch estou e o que mudou?
> `git log --oneline --graph --all`: como está o histórico?
> `git diff`: o que exatamente mudou no arquivo?

As mensagens do Git podem aparecer em inglês ou em português, dependendo da configuração do seu computador. Os exemplos deste roteiro estão em inglês. No Windows, o primeiro git add pode mostrar o aviso "LF will be replaced by CRLF": ele é normal e pode ser ignorado.

Etapa 1 de 12 · Preparar o Git

Instruções

1. Abra o terminal. No Linux e no macOS, use o terminal padrão do sistema. No Windows, use o Git Bash: no VS Code, clique na seta ao lado do + do terminal e escolha Git Bash. (O PowerShell, que abre por padrão, não aceita alguns comandos deste roteiro, como o ls -a.)
2. Confira se o Git está instalado e qual é a versão. No Linux, se ele não estiver instalado, use o gerenciador de pacotes da sua distribuição (no Ubuntu e no Debian: sudo apt install git).
3. Informe seu nome e e-mail. Eles vão aparecer como autor de cada commit que você fizer.

```
git --version
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
git config --global --list
```

> **ATENÇÃO**
>
> Use no user.email o mesmo e-mail da sua conta do GitHub. Sem isso, seus commits não aparecem ligados ao seu perfil e o professor não consegue identificar a autoria.

> **CONCEITO**
>
> O --global grava a configuração para todos os repositórios do seu usuário. Você só precisa fazer isso uma vez por computador.

Etapa 2 de 12 · Criar o repositório local

Instruções

1. Crie e inicialize um repositório com a branch principal chamada master.
2. Entre na pasta e liste todos os arquivos, inclusive os ocultos.
3. Pergunte ao Git como está o repositório.

```
git init -b master sujeito-de-sorte
cd sujeito-de-sorte
ls -a
git status
```

O que você deve ver

```
Initialized empty Git repository in .../sujeito-de-sorte/.git/

.  ..  .git

On branch master
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

CONCEITO

Um repositório é uma pasta comum com uma subpasta oculta chamada .git. É nela que o Git guarda todo o histórico: commits, branches e tags. Apagar a pasta .git transforma o repositório de volta em uma pasta comum, sem histórico. Nunca mexa nela à mão.

ATENÇÃO

Se o terminal recusar a opção -b, sua versão do Git é anterior à 2.28. Use git init sujeito-de-sorte e, depois do primeiro commit, git branch -m master.

Etapa 3 de 12 · As três áreas do Git e o primeiro commit

> **CONCEITO**
>
> O Git trabalha com três áreas. Você edita arquivos no diretório de trabalho, escolhe com git add o que vai entrar no próximo commit (staging area) e grava esse conjunto no histórico com git commit.

```
Diretório de trabalho --git add--> Staging area --git commit--> Repositório
(seus arquivos)                     (preparado)                  (.git)
```

3a · Criar o arquivo

Dentro do repositório, crie o arquivo cancao.txt no seu editor e cole a letra abaixo, uma linha por verso, sem linhas em branco no início. Salve e rode git status.

```
Presentemente, eu posso me
Considerar um sujeito de sorte
Porque apesar de muito moço
Me sinto são, e salvo, e forte
E tenho comigo pensado
Deus é Brasileiro e anda do meu lado
E assim já não posso sofrer
No ano passado
Tenho sangrado demais
Tenho chorado pra cachorro
Ano passado eu morri
Mas esse ano eu não morro
Tenho sangrado demais
Tenho chorado pra cachorro
Ano passado eu morri
Mas esse ano eu não morro
Ano passado eu morri
Mas esse ano eu não morro
Ano passado eu morri
Mas esse ano eu não morro
```

```
git status

Untracked files:
        cancao.txt
```

O arquivo existe na pasta, mas o Git ainda não o acompanha: ele está *untracked*.

3b · Colocar na staging area

```
git add cancao.txt
git status

Changes to be committed:
        new file:   cancao.txt
```

3c · Fazer o commit e ver o histórico

```
git commit -m "Texto inicial"
git log
git log --oneline
```

> **PERGUNTA INVESTIGATIVA**
>
> *O git log mostra um código longo ao lado de cada commit (ex.: bbf2e69…). O que ele identifica? Por que o git log --oneline mostra só os primeiros caracteres?*

Etapa 4 de 12 · Ver e desfazer mudanças

Instruções

1. No editor, troque a primeira palavra da linha 1 do arquivo por "Hoje" e salve.
2. Veja o que o Git detectou e o que mudou exatamente.
3. Desfaça a alteração, voltando o arquivo para o último commit.

```
git status --short
 M cancao.txt

git diff
-(linha 1 original)
+(linha 1 começando com "Hoje")

git restore cancao.txt
git status
nothing to commit, working tree clean
```

CONCEITO

O git diff mostra o texto real das linhas (no exemplo acima, elas estão descritas entre parênteses). Linhas com - foram removidas e linhas com + foram adicionadas. Uma alteração aparece como uma remoção seguida de uma adição. O git restore descarta as alterações que ainda não entraram em um commit. Use com cuidado: o que não foi commitado não tem volta.

PERGUNTA INVESTIGATIVA

Se você tivesse rodado git add antes do git restore, a alteração ainda seria descartada? Teste e descubra qual comando o git status sugere nesse caso.

Etapa 5 de 12 · Publicar o repositório no GitHub

CONCEITO

Até aqui o repositório só existe na sua máquina. Um repositório remoto é uma cópia dele hospedada em um servidor, como o GitHub. Ele permite trabalhar em equipe e manter o histórico guardado fora do seu computador. Por convenção, o remoto principal se chama origin.

Instruções

1. No GitHub, crie um repositório **público** chamado sujeito-de-sorte, **sem** README, .gitignore ou licença. Ele precisa nascer vazio.
2. Copie o endereço do repositório (a URL da página): é esse link que você vai entregar ao professor.
3. Conecte o repositório local ao remoto e envie a master.
4. Atualize a página do GitHub e confira se o arquivo e o commit apareceram.

```
git remote add origin https://github.com/SEU_USUARIO/sujeito-de-sorte.git
git remote -v
git push -u origin master
```

CONCEITO

git push envia seus commits para o remoto. O -u liga a branch local à remota; depois disso, nessa branch, basta git push.

ATENÇÃO

No primeiro push, o Git pede autenticação. No Windows, normalmente abre o navegador para você entrar na sua conta do GitHub. Se pedir usuário e senha no terminal (o que é comum no Linux), a senha da conta não funciona: use um token de acesso pessoal. Crie um token clássico em GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic), marque o escopo repo e cole o token no lugar da senha. Use o token clássico, e não o "fine-grained": este não permite enviar para o repositório do colega na atividade em dupla. Para não digitar o token a cada push, rode git config --global credential.helper 'cache --timeout=7200' (ele fica guardado na memória por 2 horas).

Etapa 6 de 12 · Branches: develop e a primeira feature

> **CONCEITO**
>
> Uma branch é uma linha de desenvolvimento independente. Tecnicamente, é só um ponteiro para um commit, que avança a cada novo commit feito nela. Neste roteiro usamos uma política comum: a master guarda as versões publicadas, a develop integra o trabalho em andamento e cada funcionalidade nasce em uma branch feature/… própria.

Instruções

1. A partir da master, crie a develop, entre nela e envie-a para o GitHub.
2. A partir da develop, crie a feature/cabecalho e liste as branches. O asterisco indica em qual você está.
3. Na feature, acrescente no topo do arquivo o cabeçalho indicado abaixo e faça commit.

```
git switch -c develop
git push -u origin develop

git switch -c feature/cabecalho
git branch

# no editor, as 3 primeiras linhas do arquivo passam a ser:
#   SUJEITO DE SORTE
#   Belchior (1976)
#   (linha em branco)

git commit -am "Adiciona título e autor"
```

git commit -am faz o add e o commit de uma vez, mas só para arquivos que o Git já acompanha. Arquivos novos ainda precisam de git add.

Etapa 7 de 12 · Primeiro merge: fast-forward

Instruções

1. Volte para a develop. Abra o arquivo: o cabeçalho sumiu, porque ele só existe na feature.
2. Integre a feature/cabecalho na develop e leia a mensagem do Git.
3. Veja o histórico e envie a develop e a feature para o GitHub.

```
git switch develop
git merge feature/cabecalho
git log --oneline --graph --all

git push
git push -u origin feature/cabecalho
```

CONCEITO

O Git respondeu "Fast-forward": como a develop não tinha nenhum commit novo desde que a feature foi criada, ele apenas avançou o ponteiro da develop até o commit da feature. Nenhum commit novo foi criado e o histórico continua uma linha reta. Esta é a única vez que uma feature entra na develop pelo terminal. Daqui em diante, as integrações na develop serão por Pull Request, e o botão de merge do GitHub não faz fast-forward: ele sempre cria commits novos.

PERGUNTA INVESTIGATIVA

O que precisaria ter acontecido na develop para que o Git NÃO conseguisse fazer fast-forward?

Etapa 8 de 12 · Três features em paralelo

Agora simule três pessoas trabalhando ao mesmo tempo, todas partindo do mesmo ponto da develop. Crie as três branches antes de editar qualquer coisa.

```
git branch feature/versao-vencer
git branch feature/versao-crescer
git branch feature/versos-novos
```

8a · feature/versao-vencer

Altere SOMENTE a linha 4 do arquivo (o primeiro verso, logo abaixo do cabeçalho): troque a última palavra da linha por "vencer". As demais linhas ficam como estão.

```
git switch feature/versao-vencer
# linha 4: última palavra trocada por "vencer"
git commit -am "Altera linha 4: versão vencer"
git push -u origin feature/versao-vencer
```

8b · feature/versao-crescer

Altere a MESMA linha 4, com outro conteúdo: troque a última palavra por "crescer".

```
git switch feature/versao-crescer
# linha 4: última palavra trocada por "crescer"
git commit -am "Altera linha 4: versão crescer"
git push -u origin feature/versao-crescer
```

8c · feature/versos-novos

Acrescente quatro versos seus no FINAL do arquivo, em dois commits: primeiro as linhas, depois um pequeno ajuste.

```
git switch feature/versos-novos
# acrescente ao final, por exemplo:
#   A vida tem um repositório,
#   E o merge traz um conflito;
#   Os commits que nós criamos
#   Ficam no histórico, está escrito.
git commit -am "Adiciona versos no final"
# ajuste algo (ex.: "está escrito" → "fica registrado")
git commit -am "Ajusta última linha"
git push -u origin feature/versos-novos
```

REPARE

Ao trocar de branch com git switch, o conteúdo do arquivo muda no editor: cada branch tem a sua versão. Rode git log --oneline --graph --all e veja que a develop está parada e três ramos saem dela. No GitHub, a lista de branches (botão "branches" na página do repositório) mostra as mesmas branches.

Etapa 9 de 12 · Pull Request com merge commit

> **CONCEITO**
>
> Um Pull Request (PR) é um pedido para integrar uma branch em outra. Nele, o time revisa as alterações e decide como fazer o merge. É assim que se trabalha na maioria dos projetos reais.

Instruções

1. No GitHub, em Settings → General → Default branch, troque a branch padrão para develop, para que os Pull Requests já apontem para ela. Depois, abra um Pull Request da feature/versao-vencer para a develop (aba Pull requests → New pull request). Confira se a base é a **develop**.
2. Na descrição do PR, responda a pergunta investigativa da Etapa 7. Depois, veja a aba "Files changed": é o mesmo git diff que você viu no terminal.
3. Integre com o botão **"Create a merge commit"**. Não apague a branch depois do merge.
4. Atualize sua develop local e veja o histórico.

```
git switch develop
git pull
git log --oneline --graph --all
```

> **CONCEITO**
>
> O botão "Create a merge commit" faz o mesmo que git merge --no-ff: cria um commit de merge com dois "pais", o último commit da develop e o último da feature. Os commits da feature entram no histórico, e fica registrado que existiu uma branch e quando ela foi integrada.

> **ATENÇÃO**
>
> Depois de cada merge feito no GitHub, sua develop local fica desatualizada. Sempre rode git pull na develop antes de continuar.

Etapa 10 de 12 · Conflito no Pull Request

Instruções

1. Abra um Pull Request da feature/versao-crescer para a develop. O GitHub vai avisar: "This branch has conflicts that must be resolved". Na descrição do PR, explique com suas palavras por que o GitHub acusou conflito.
2. **Não** use o botão "Resolve conflicts" do site. Resolva no terminal, trazendo a develop atualizada para dentro da feature.

```
git switch feature/versao-crescer
git fetch origin
git merge origin/develop

CONFLICT (content): Merge conflict in cancao.txt
Automatic merge failed; fix conflicts and then commit the result.

git status
        both modified:   cancao.txt
```

O que aparece no arquivo

```
<<<<<<< HEAD
(linha 4 terminando em "crescer")
=======
(linha 4 terminando em "vencer")
>>>>>>> origin/develop
```

- Entre <<<<<<< HEAD e =======: o que está na branch atual (a sua feature, versão crescer).
- Entre ======= e >>>>>>>: o que vem da develop do GitHub (versão vencer, integrada na Etapa 9).

Resolva o conflito

1. Decida como a linha deve ficar: com uma versão, com a outra ou com as duas combinadas (por exemplo, terminando em "vencer e crescer").
2. Apague as três linhas de marcadores. O arquivo precisa ficar sem nenhum <<<<<<<, ======= ou >>>>>>>.
3. Salve, marque o arquivo como resolvido, conclua o merge e envie a feature.

```
git add cancao.txt
```

```
git commit -m "Resolve conflito na linha 4"
git push
```

Conclua o Pull Request

4. Volte ao Pull Request: o aviso de conflito sumiu. Integre com "Create a merge commit".
5. Atualize a develop local com git switch develop e git pull.

> **CONCEITO**
>
> Um conflito acontece quando duas branches alteram a mesma região de um arquivo a partir do mesmo ponto de partida. O Git não tem como saber qual versão é a certa, então para e pede que uma pessoa decida.
>
> git fetch baixa as novidades do remoto sem alterar suas branches, e origin/develop é a cópia local da develop do GitHub. No dia a dia, quem resolve o conflito é quem abriu o PR, dentro da própria feature, e a develop só recebe o código já resolvido.

> **SE VOCÊ SE PERDER**
>
> git merge --abort cancela a tentativa de merge e devolve a feature ao estado anterior. Depois é só tentar de novo.

> **PERGUNTA INVESTIGATIVA**
>
> *A feature/versos-novos também alterou o cancao.txt e ainda não foi integrada. Você espera que o Pull Request dela acuse conflito? Anote sua aposta antes da próxima etapa.*

Etapa 11 de 12 · Squash pelo Pull Request

Instruções

1. Abra um Pull Request da feature/versos-novos para a develop. Na descrição, registre a aposta que você fez no fim da Etapa 10, antes de ver o resultado.
2. Na seta ao lado do botão de merge, escolha **"Squash and merge"** e confirme.
3. Atualize a develop local, veja o histórico e a lista de branches já mescladas na develop.

```
git switch develop
git pull
git log --oneline --graph --all
git branch --merged develop
```

CONCEITO

O botão "Squash and merge" faz o mesmo que git merge --squash seguido de um commit: cria um único commit novo com o resultado de todos os commits da feature. O histórico da develop fica mais enxuto, mas os commits individuais da feature não entram nele.

PERGUNTA INVESTIGATIVA

No GitHub, o PR da feature/versos-novos aparece como "Merged". Ela aparece na lista do git branch --merged develop? Por que o GitHub e o Git "discordam"?

Etapa 12 de 12 · Publicar uma versão na master

Instruções

1. Abra um Pull Request da develop para a master, com o título "Release 1.0". Na descrição, responda a pergunta investigativa da Etapa 11. Depois, integre com "Create a merge commit".
2. Atualize a master local, crie a tag e envie-a para o GitHub.
3. No GitHub, abra Insights → Network e confira o desenho completo do histórico.

```
git switch master
git pull
git tag v1.0
git push origin v1.0
git log --oneline --graph --all --decorate
```

CONCEITO

Uma tag é um nome fixo para um commit, usado para marcar versões (v1.0, v1.1…). Diferente de uma branch, ela não avança quando novos commits são feitos. O git push não envia tags: elas precisam ser enviadas uma a uma (git push origin v1.0) ou todas juntas (git push --tags).

Atividade em dupla

Até aqui só você mexeu no seu repositório. Agora vocês vão ver um conflito acontecer entre duas pessoas que trabalham no mesmo remoto. Desta vez, os dois vão commitar direto na develop, sem Pull Request. Isso é proposital, porque é o jeito mais simples de ver o GitHub recusar um push. No dia a dia, continue usando features e Pull Requests.

1. O aluno A adiciona o aluno B como colaborador do seu repositório.
2. O aluno B aceita o convite (pelo e-mail ou em github.com/notifications) e clona o repositório. Sem aceitar o convite, o push dele falha por falta de permissão.
3. Os dois alteram, cada um na sua máquina e na develop, a MESMA linha do arquivo (por exemplo, a linha 15) e fazem commit.
4. O aluno A faz push primeiro. O push do aluno B será rejeitado.
5. O aluno B traz as alterações do A, resolve o conflito como na Etapa 10 e faz push.

```
# aluno B
git clone https://github.com/USUARIO_DO_A/sujeito-de-sorte.git
cd sujeito-de-sorte
git switch develop

# depois do push rejeitado
git fetch origin
git merge origin/develop
# resolva o conflito e depois:
git add cancao.txt
git commit -m "Resolve conflito com o colega"
git push
```

> **PERGUNTA INVESTIGATIVA**
>
> *Por que o Git rejeitou o push do aluno B em vez de simplesmente sobrescrever o que o aluno A tinha enviado?*

Checklist de entrega

Antes de enviar o link, confira no GitHub se o seu repositório tem tudo o que o professor vai verificar:

- Commits com o seu nome e ligados ao seu perfil do GitHub (configuração da Etapa 1).
- As branches develop, feature/cabecalho, feature/versao-vencer, feature/versao-crescer e feature/versos-novos, nenhuma apagada.
- Três Pull Requests integrados na develop: versao-vencer (merge commit), versao-crescer (conflito resolvido e merge commit) e versos-novos (squash).
- As respostas pedidas na descrição de cada Pull Request (Etapas 9 a 12).
- O Pull Request "Release 1.0" integrado na master.
- A tag v1.0 visível no repositório (em Tags).
- O repositório público: abra o link em uma janela anônima do navegador e confira se ele aparece sem login.

Perguntas para refletir (não é necessário entregar)

1. Explique a função de cada uma das três áreas do Git (diretório de trabalho, staging area e repositório) usando o que você viu na Etapa 3.
2. Com base no seu histórico, explique a diferença entre o merge da Etapa 7 (fast-forward no terminal), o da Etapa 9 ("Create a merge commit") e o da Etapa 11 ("Squash and merge").
3. Por que as features das duas versões entraram em conflito, mas a versos-novos não, se as três alteraram o mesmo arquivo?
4. Quem decidiu o conteúdo final da linha em conflito: o Git ou você? O que isso significa quando o conflito acontece entre duas pessoas de um time?
5. Imagine que as duas features da linha 4 tivessem ficado três semanas em suas branches, enquanto dezenas de linhas eram alteradas na develop. Relacione esse cenário ao "merge hell" e à Integração Contínua (Cap. 10).
6. Na política usada aqui (master, develop, features), qual branch representa o que está "em produção"? Em que momento uma alteração chega até ela?
7. Por que o conflito da Etapa 10 foi resolvido dentro da feature, e não na develop? Que vantagem isso traz quando várias pessoas trabalham no mesmo repositório?

Resumo dos comandos

Comando	Para que serve
`git config --global user.name / user.email`	Define o autor dos seus commits
`git init -b master <pasta>`	Cria um repositório local
`git status`	Mostra a branch atual e o estado dos arquivos
`git add <arquivo>`	Coloca alterações na staging area
`git commit -m "msg"`	Grava a staging area no histórico
`git commit -am "msg"`	add + commit de arquivos já acompanhados
`git log --oneline --graph --all`	Mostra o histórico em forma de grafo
`git diff`	Mostra o que mudou e ainda não foi commitado
`git restore <arquivo>`	Descarta alterações não commitadas
`git branch / git branch <nome>`	Lista branches / cria uma branch
`git switch <nome> / git switch -c <nome>`	Troca de branch / cria e troca
`git merge <branch>`	Integra uma branch (fast-forward quando possível)
`git merge --abort`	Cancela um merge com conflito
`git remote add origin <url>`	Conecta o repositório local a um remoto
`git remote -v`	Lista os remotos configurados
`git push -u origin <branch>`	Envia uma branch e a liga à remota
`git push`	Envia os commits da branch atual
`git pull`	Traz e integra as novidades do remoto
`git fetch origin`	Baixa as novidades sem alterar suas branches
`git merge origin/develop`	Integra a develop do remoto na branch atual

`git clone <url>`	Copia um repositório remoto para sua máquina
`git tag <nome>`	Marca um commit com um nome de versão
`git push origin <tag>`	Envia uma tag para o remoto